

In JavaScript, an object is a collection of related data and functionality stored as key-value pairs. Keys are strings or symbols (property names), and values can be any data type, including functions, which are then called methods.

```
const user = {  
  name: "John", // Property (String)  
  age: 30, // Property (Number)  
  greet: function() { // Method  
    console.log("Hello, my name is " + this.name);  
  }  
};
```

```
// Accessing data  
console.log(user.name); // "John"  
user.greet(); // "Hello, my name is John"
```

Accessing properties and methods in JavaScript is quite intuitive once you understand, that an object is essentially a collection of keys and values. There are two primary ways to access these: Dot Notation and Bracket Notation.

```
const car = {  
  brand: "Tesla",  
  model: "Model 3",  
  "battery capacity": "75 kWh", // Property with a space  
  start: function() {  
    return "The engine is purring (or humming)!";  
  }  
};
```

```
// --- Accessing Properties ---  
console.log(car.brand); // "Tesla" (Dot notation)  
console.log(car["model"]); // "Model 3" (Bracket notation)  
console.log(car["battery capacity"]); // "75 kWh" (Brackets required for spaces)
```

In JavaScript, the `this` keyword is a reference to the object that is currently executing the code. Unlike many other languages where this always points to the instance of a class, in JavaScript, its value is determined by how a function is called (runtime binding), not where it is defined.